

Lightweight Fault Detection and Localisation in AWS Serverless Microservices: Accuracy and Monitoring Overhead against Deep-Learning Baselines

Yashaswini Penumarthi

24262404

MSc in Cloud Computing

National College of Ireland

Abstract

The most accurate published detectors of microservice faults achieve their accuracy by training deep models on large labelled telemetry corpora. Those are not just expensive in a serverless environment without the ability to train compute, but they're not possible in such an environment. The accuracy that a rule-based alternative sacrifices and the monitoring expenses are not measured. This proposal details a controlled evaluation that uses a threshold and deviation rules based on CloudWatch telemetry and order dependency ranking based on X-Ray to evaluate injected faults in an order processing microservice. Precision, Recall, F1 score, detection delay and top-k localisation were used to measure accuracy in the form of precision and recall, while overhead was measured as volume of telemetry, added latency and cost. The accuracy conceded is measured, not asserted, as it is provided by reproducible baselines on a published benchmark.

Keywords: serverless observability, fault localisation, AWS X-Ray, monitoring overhead, anomaly detection

1. Introduction

In microservice systems, faults are propagated. One service will report high latency, which is likely to be caused by another service's failure; this phenomenon will manifest as a timeout in the other service. To fill the gap, observability tooling has been developed, and AWS has two managed sources that provide the signal: CloudWatch, which tracks metrics and logs, and X-Ray, which tracks a request through the services it traverses.

Learning is the most accurate published methods to exploit that signal. Xing et al. (2025) test the detection accuracy to be about 93 per cent and top-three accuracy as 97 per cent on localisation, with the causal inference deep model comprising two channels. Both Zhang, J. and Yang (2026) achieve comparable accuracy with only processors and Zhang, W. et al. (2026) use a dependency graph with the help of language models to localise root causes.

With each of those results there is a setting that it is true when and a setting that it is false when. A supervised detector has to be trained, and training entails the use of labeled examples of the failures that are to be detected. There is no such history with a serverless service deployed last month, no labels in the service, and hours of training compute is usually not covered in the cost ceiling. The barrier is not cost, it is data availability, and nothing can be done to eliminate the barrier.

The only alternative that is available in that setting is the untrained alternative, fixed thresholds and deviation from a short rolling baseline, with the dependency ordering implicit in a trace. There is no need for a history of such rules. They have not been compared at all on terms of accuracy or volume, latency, and costs of continuous tracing and logging, with the learned methods.

The question this study asks is then: Where does rule-based fault detection and localisation fit the accuracy overhead spectrum, compared to learned baselines, in serverless microservices?

The question is not just a one time deployment. Prior to any failure history, a team setting up a small service is left to choose the level of observability they want to instrument, with no evidence of how much this may or may not affect the accuracy of their detection, nor the overheads they experience. The research community, on the other hand, rates the detection accuracy without considering the monitoring cost, and these two are measured separately in the reviewed literature.

It leads to four objectives: The first measures the detection accuracy when faults are injected to the system, i.e., precision, recall, F1 and detection delay. The latter measures localisation accuracy using the top-k accuracy and mean rank of the true root-cause service. The third is a measure of the monitoring overhead it adds, in terms of telemetry volume, added latency and estimated cost. The fourth compares the outcome to baseline learning outcomes that have been learned on the same set of data.

This question cannot be answered on the basis of existing work because there is no possibility of assessing the two concepts of accuracy and overhead against each other on equivalent terms. Learned methods report accuracy when they are used in data conditions the target setting cannot provide; benchmarks provide a means, but do not provide a position; untrained trace analysis provides the feasibility, but no means; untrained trace analysis provides a means, but not a position. This study therefore reports the two dimensions together, but measures them on one design.

From a practical standpoint the study provides an accuracy-overhead position that a team can consider before they invest in instrumentation, and no one of the reviewed work can offer. The methodological result is a comparison protocol, which makes it possible to see an untrained method as trained, and a trained one as untrained, a contribution because they cannot be executed on the same input.

The rest of this is structured as follows. In Section 2, the detection literature is revisited in three stages, that is, the accuracy ceiling is posed by learned methods, which require a labelled corpus; untrained, threshold-based methods set the stage for feasibility, and at times even parity, without paying the price for what they observe; finally, a published benchmark provides the substrate on which the two can be compared, with the comparison itself not performed. Section 3 outlines the two-arm design, the injection procedure and data sets, the three-leg comparison protocol, the hypotheses, and the tests to determine the hypotheses, the ethical position, and the twelve week schedule. The works cited are presented in Section 4.

2. Literature Review

A. *Learned Detection and Localisation in Microservices*

Learned detection has come to focus on the accuracy while also splintering on cost. In this paper, Xing et al. (2025) fuse two telemetry channels and employ causal inference, reporting an F1 of nearly 93 per cent upon detection and comparable top-three localisation. In Zhang, J. and Yang (2026), the accuracy goal is the same, but there is an explicit constraint on the number of processors used, and the dynamic graph networks and recurrent modelling are done with only processor resources. The first is optimizing for accuracy and will bear the burden of training; the second is optimizing the burden of hardware and keeping the model.

If the model is retained, so is its precondition. Both approaches demand a labeled corpus of previous failures as a learning base for what a failure looks like, and Zhang, J. and Yang (2026) note that adding signals that are different can add feature redundancy and modelling complexity. The number of computes that are not used to train does not reduce the number of history to train on, which is the constraint that will bind in a new service deployed.

Reported accuracy is also dependent on the setting of the decision boundary. Liu et al. (2023) test multivariate detection on production cloud metrics, with an average F1 score of 0.9494 compared to 0.8892 for the strongest deep baseline but report that thresholds are swept manually, and that the threshold selection of the baselines is replaced by the same process. A part of this separation comes from tuning the boundary, not the underlying model, and that's significant because the quantity they tune by hand is the quantity a rule-based arm must fix ahead of time, and never modify.

The same boundary shown from a different angle in a graph-based localisation. In smaller systems, Zhang, W. et al. (2026) build a dependency graph and use a language model to aid recovery, with awareness that for larger systems, scalability is an issue. The structure that they use is not learned, but the language-model component adds an additional per-invocation cost that is not reported in the evaluation.

Learned methods thus set the accuracy limit and its cost in terms of data. Given that the research question is: what accuracy remains when that data is not available, the untrained alternatives are provided.

B. *Lightweight Alternatives and the Cost of Observing*

Without reporting accuracy without reporting overhead, you're withholding half of the decision. A team setting up a small service is simultaneously instrumenting it to detect and telemetrically emitting, while neither Xing et al. (2025) nor Zhang, W. et al. (2026) report the second quantity.

The size of the accuracy premium is also subject to debate. Schmidl, Wenig and Papenbrock (2022) implement seventy-one detection algorithms across 976 time series and determine that the optimal cost-benefit detector is an un-trained statistical model that achieves an average AUC-ROC of 0.83 with a time of 2.2 milliseconds per point—showing that deep learning is yet to be competitive at the level of training cost. This indicates that the premium could be reduced compared to the precondition it imposes, which is assumed in the figure of Set al. (2025). Their series, however, are generally univariate, rather than correlated, cloud metrics, and AUC-ROC is threshold-free, meaning they don't settle in on an operating threshold.

El Khairi et al. (2024) instead do not train anything, but rather use peers of the same service to derive a baseline at runtime, and mark a snapshot as "dissimilar" if it is not close enough to the peers than a threshold set by sweep; they report an average precision of 91.08 per cent and an average recall of 98.35

per cent that are stable across updates that degrade trained baselines. This is clear counter-evidence to the assumption that untrained detection has to give way to accuracy and a template for the calibration of a threshold rather than its assertion. It assumes that there are multiple concurrent similar copies, which can't be guaranteed by independent scaled serverless instances, and it does not rank the root causes nor charge the observation.

Even though it is not trained, a type of localisation is shown to be viable. Zhao Zhang et al. (2024) localise performance problems by first identifying common attribute combinations in abnormal traces and uncommon ones in normal traces without training any model, and then validate their work at a major software engineering conference. Their approach is based only on traces, though, and is not reported to have incurred any monitoring overhead, nor was its accuracy compared against learned detectors; thus, the methodology they implemented leaves their approach at the same place as that of El Khairi et al. (2024): viability shown, position not established.

The comparison those studies leave open has become recently tractable. Pham et al. (2025) share a benchmark of 735 labelled failure cases for three microservice systems, along with metrics, logs and traces for each and 15 reproducible baselines. Together, Zhang et al. (2024) and Pham et al. (2025) have each half of the answer, and neither accomplishes the goal of a position by itself: a comparison that isn't accompanied by a study won't produce a position, and a study by itself, without a comparison, won't produce one.

Table 1 lists the reviewed approaches and how far they are from this proposal. Having treated the problem of viability and comparability separately in the literature, the four directions that leave the question unanswered are summarized together in the next section.

Table 1 Reviewed approaches contrasted with the proposed evaluation

Study	Approach	Evaluation methodology	Strengths	Limitations	How this proposal differs
Xing et al. (2025)	Dual-channel deep learning with causal inference	Trained on labelled telemetry; F1 and top-3 localisation reported	Highest published accuracy	Requires labelled history and training compute absent in new serverless services; no overhead reported	Measures accuracy attainable without any training history, and the overhead it costs
Zhang, J. and Yang (2026)	Dynamic graph networks with recurrent modelling	Processor-only spatiotemporal detection	Removes the accelerator requirement; notes signal redundancy	Still a learned model requiring a labelled corpus	Requires no corpus at all
Zhang, W. et al. (2026)	Graph-based localisation with language-model assistance	Dependency-graph construction and assisted recovery	Effective in smaller systems	Assisting model adds unreported per-invocation cost; scalability limits acknowledged	Uses dependency ordering only, and prices the observation
Zhang, C. et al. (2024)	Attribute-based trace localisation	Search for attribute combinations frequent in anomalous traces	Learning-free; validated at a flagship venue	Traces only; no accuracy comparison against learned methods; no overhead figure	Adds metrics and logs, compares against learned baselines, and reports overhead
Pham et al. (2025)	Benchmark and evaluation harness for root-cause analysis	735 labelled failure cases, three systems, fifteen reproducible baselines	Enables like-for-like comparison on identical inputs	Supplies data and baselines rather than findings; no cloud cost or latency dimension	Uses it as the comparison substrate and adds the overhead measurement it cannot provide

C. *Why This Research Is Necessary*

There are four ways to think about this problem that have been discussed in the past with none of them getting the measurement that is sought in the question. As shown by Xing et al. (2025) and Zhang, J. and Yang (2026) respectively, deep-learning detection offers the highest accuracy but under settings that the target setting cannot achieve. Untrained and threshold-based methods set the feasibility; in Schmidl, Wenig and Papenbrock (2022) and El Khairi et al. (2024) something close to parity, without putting themselves against the learned accuracy or pricing the observation; Liu et al. (2023) show how much of the apparent gap is tuning the threshold. Many of the reported per-invocation costs are avoided in Zhang, W. et al. (2026) and the comparison is taken away, whereas Zhang, C. et al. (2024) sacrifice the comparison for some of the reported per-invocation costs. Benchmark infrastructure of Pham et al. (2025) provides labelled failures and reproducible baselines, while Mertz and Nunes (2023) and Huang et al. (2024) report on observation without any relation to any detection result.

The implication on the other hand is that it is impossible for a team to determine what accuracy they give up if they instrument rules rather than train a model, or what the cost of the two is to run. This proposal will fill this gap with controlled fault injection, evaluation with reproducible baseline data on common data, and pre-committed statistical analysis. The niche that this leaves is stated exactly in the next subsection.

D. *Research Niche*

There is not a single study that evaluates an untrained detection method, compares it to learned baselines on common data and reports the monitoring overhead incurred, that is all at once. None of the studies reviewed meet all four: Xing et al. (2025) report detection and localisation accuracy, but do not compare to an untrained method, and do not report overhead; while Pham et al. (2025) provide common data and baselines, but do not measure overhead or report a study. The instrumentation switch controls all four simultaneously. Each of the designs reviewed is effective at its own purpose; none is designed to provide answers for this one, nor does any record the measurements required for answering it.

The base of this research is on the study of Xing et al (2025). Because a new and cost constrained service cannot provide a failure history to label, and that lack is the issue addressed here, their accuracy is achieved by training them on a labelled telemetry corpus that cannot be obtained from such a service. This study quantifies this concession to accuracy in the F1 and top-k metrics, measuring the same on the same terms, and also reports the overhead associated with the monitoring.

The contribution is a jointly reported accuracy-overhead location for untrained detection and localisation of serverless microservices, pair which no examined study carries out, along with the protocol that provides comparable location for untrained and trained method at all. Taken together, the results presented above are consistent with three expectations that can be framed as thresholds that a measurement can violate: the rule-based arm doesn't lose more than 10 percentage points when compared to the most reproducible baseline; the rule-based arm loses more than 10 percentage points on top three localisation accuracy; the rule-based arm reduces the amount of telemetry by at least 50%. By reserving them beforehand they remain defensible and the accuracy overhead position has either a positive or negative run depending on which direction each it runs in.

Practitioners can be equipped with evidence of how different types of instrumentation relate to different types of detection results prior to the occurrence of a failure. The research community benefits from a

common measured accuracy-overhead baseline to which learned detectors can be compared, on a metric they have not been reporting. This niche is filled by the evaluation described in Section 3. Multi-service topologies, online rule adaptation and hybrid rule and model detection and cross-provider replication are deferred to future work.

3. Research Method & Specification

A controlled evaluation with two detection arms answers this question. An order-processing microservice is instrumented, faults are injected under a controlled script, and the resulting telemetry is analysed by a rule-based arm and compared against reproducible learned baselines. Figure 1 shows the arrangement and the experimental workflow.

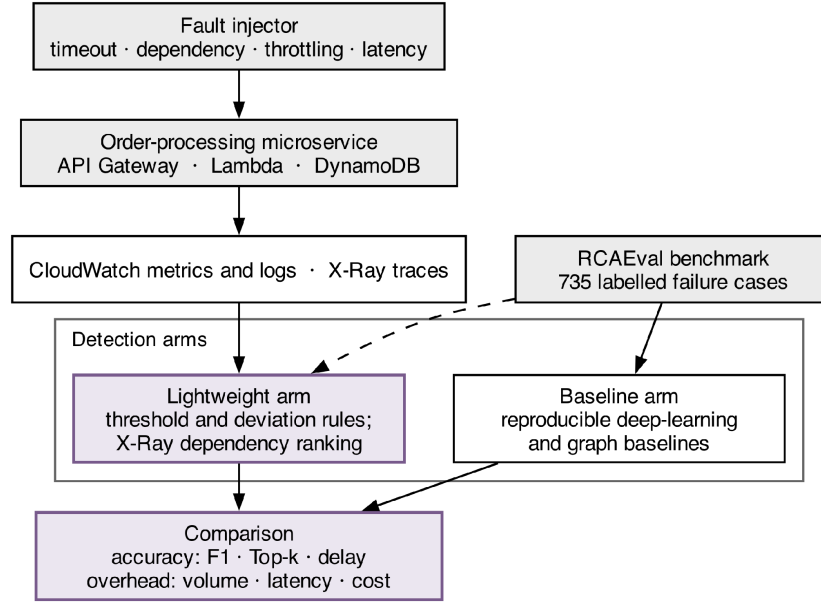


Figure 1 Experimental setup and comparison workflow

A. Research Method

The independent variables are the detection method, which is at two levels (the rule-based arm and the learned baseline arm); the fault type, which is at four levels (timeout, dependency failure, throttling and elevated latency); and the load level, which is at two levels (steady baseline rate and a peak rate five times the baseline), added because a detector's false-positive behaviour under load is not the same as its behaviour at rest. The system under test is a single order-processing microservice composed of an API Gateway endpoint, serverless functions and a managed table. The rule-based arm only is specified concretely and not left as a category. The following three CloudWatch metrics are monitored for each service: invocation error count, duration, and throttle count. A detection is triggered if a metric is more than 3 standard deviations above or below a rolling 30 minute baseline calculated from the preceding 30 minutes of measurements, or if it exceeds a fixed threshold based on the 99th percentile of the calibration period, whichever threshold is met first. Base line windows and thresholds are calculated in 24 hours during a fault free period and once baseline is calculated, it cannot be adjusted. Localisation ranks each service that is listed in the traces that cover the detection window using a combination of the proportion of times it is listed in the failed traces, and how deep it is in the dependency graph, so that services that are both

frequently listed and close to the observed symptom rank highest; that is how top-k accuracy and mean rank are computed.

There are two families of dependent variables. For accuracy, precision, recall, F1 and detection delay are used for the detection; top-k accuracy and mean rank of the true root-cause service are used for localisation. Emitted log and trace volume, added end-to-end latency, estimated monitoring cost are used as indications of overhead. According to Mertz and Nunes (2023), this is a matter of materials and policy dependence; they have found the throughput reductions for uniform sampling to be between 13.5 to 23.1 per cent and for adaptive sampling to be between 4.6 to 11.6 per cent, with the sampling policy indicated by each number. They also demonstrate that there is no inherent trade-off between telemetry volume and diagnostic quality, by showing that an average increase of 32.63 per cent in top-one root-cause accuracy is achieved with a small fraction of traces while reducing the monitoring cost, for a state-aware sampler. F1 and top-k are calculated on the same metrics as Xing et al. (2025), and that is how the untrained result is compared to the learned ceiling.

Fault labels are automatically ground truth: the injector knows what and when it injected, and all detections are scored without manual annotation. The injected taxonomy aligns to the fault categories published with the benchmark to ensure a degree of commensurability between the two comparison legs.

Arms are all identical with respect to region, account, service topology, workload generator and rule thresholds. The first problem is the true variability of the platform, because an injected fault can overlap with it and a fault-free control period will be used before each campaign to set a false-positive threshold for detections. The second is the act of tracing, which perturbs the latency measured, and not a nuisance; therefore, latency is measured with tracing both on and off, and the difference is reported as the overhead.

If thresholds are tuned using the same data that they are tested with then there is an inherent overestimate of accuracy within the system, which is why they are set on a different time period for calibration and will not be updated after that. External validity is limited to one provider, one topology and one service and larger, more varied systems are beyond the scope of these measures. The published accuracy of learned methods was measured on other datasets and is thus a ceiling accuracy measured in ideal data conditions, not a same-rig accuracy.

B. Research Resources

The experiment is located on five managed services: Lambda, API Gateway, DynamoDB, CloudWatch and X-Ray. The service and instrumentation are both declared in infrastructure code, thereby allowing all configurations to be created and destroyed exactly the same, and keeping the bill contained. Fault injection is performed on the service and dependency-access paths controlled by a scheduled script and the injection time is precisely recorded. Each fault type has a known mechanism: Timeout occurs when the dependency call is held over the client-provided timeout; Dependency failure occurs when the invoked service returns an error response; Throttling occurs when a call adds too many rows to the table to be handled by the provisioned throughput; Elevated latency occurs when a delay is introduced between the client call to the service and the return of a successful response. The injection lasts for 60 seconds and has a recovery time of 5 seconds between each injection that allows a detection to be linked to a previous fault.

The study is based on two sets of data. The first one comes from the researcher's infrastructure: synthetic order transactions with no personal, private or third party information, and with faults injected according to

the script. The second one is the published benchmark by Pham et al. (2025) that provides 735 failure cases labeled in three microservice systems and fifteen reproducible baseline for use under its published licence.

C. Evaluation

The comparison is made for three feet, for you can't just run an untrained method and a trained method on the same inputs. First, the published data from Xing et al. (2025) is reported as a maximum accuracy that could be achieved in ideal data situations, and is not used as a same-rig comparison. Second, the rule-based arm is run on the benchmark data provided by Pham et al. (2025) in addition to the baselines provided with the data, allowing for a like-for-like comparison of accuracy on the same input data. Third, the overhead size is being measured on the researcher's own AWS service, as the benchmark provides no cloud cost or latency numbers, and would not be able to provide them. That third leg is asymmetric, and is not reported as a measured difference, but rather as such. Overhead is generated directly for the rule-based arm, against a real instrumented service. It cannot be achieved with the learned baselines, where it is used to perform offline on recorded telemetry and never instruments a running system. The comparison therefore is only in absolute terms and all overhead claims are moderated by the telemetry its published inputs would require, which is an asymmetry.

The produced data set is checked in multiple different manners. Fault labels are ground truth by construction; the injected taxonomy is similar to the published taxonomy of the benchmark; and the rule-based arm's accuracy on the benchmark is sufficient to ensure the accuracy of the rule-based arm on the researcher's own rig is in a consistent range. When the two estimates differ, the difference is stated but not balanced.

Each comparative objective has a clearly stated null and alternative. Detection: H0: There is no difference between the rule based arm and the reproducible baselines on common benchmark data, H1: There is a difference. Localisation: H0 is stating that no difference between arms in top-k accuracy; H1 is stating that there is some difference between arms in top-k accuracy. Overhead: H0 says that tracing does not affect end to end latency; H1 says it does. All are non-directional because the amount of any sacrifice in terms of accuracy is precisely what is uncertain.

No test is selected until the distribution has been found normal. An independent-samples t-test is used when it holds and Cohen's d as effect size measure is used for each two-arm comparison, otherwise the Mann-Whitney U test is used instead, with rank-biserial correlation used as the effect size measure. Both detection delay and latency distributions are reported in the median and upper percentile more than the mean as both are expected to be right skewed.

Each test is evaluated under the assumption of an alpha of 0.05, with the hypotheses evaluated in a body of collected evidence, so they are adjusted by the Holm-Bonferroni method. Replication is not delayed – the 60 per arm number of injections is selected to allow a difference of 10 percentage points between F1 and F2 to be detected at 80 per cent power for a 2-sample comparison at this alpha level. Ten percentage points is also the smallest difference that is considered practically significant. The pilot verifies the variance observed during the pilot, not to determine whether the arm will be competitive or not, but to confirm that the variance observed is consistent with the calculation; the decision rule is written down prior to the start of the collection: the arm is competitive if its F1 is within 10 points of the highest reproducible baseline, and the arm reduces the amount of telemetry by at least 50 %.

D. Ethical Considerations of the Research

This study does not involve people, or personal information is not recorded by this study, all transactions are created for the experiment and therefore there is no consent or obligation to the protection of data. The benchmark data is published for research purposes and is made available under the terms of the licence, including the requirement to attribute the data.

The introduction of faults into a commercial platform which is required to receive load, has to be within the limits of acceptable use that the platform publishes. Faults are only put in place in services that the researcher controls and within the researcher's own account and at load levels kept below free-tier-safe limits; shared capacity is not stressed and so no neighbouring tenant is affected by the experiment. Compute and telemetry are both charged and the number of runs depends on the power calculation not the budget. This ceiling sets the amount of money that can be committed and, therefore, the energy and environmental impact that the study has.

Particular care must be taken in reporting where a comparison enhances the proposed method. The rule thresholds are set in advance and do not get refined during the evaluation; the published accuracy of learned methods is reported under their own data conditions and not as a defeated competitor; false positives from fault-free control are also reported along with the detections; and the single-provider/single-topology scope goes with each summary. Any platform behaviour that is revealed by the measures that the provider has not documented is reported to the provider before it is reported in the write-up. A service definition, injection scripts, configurations of rules and the measurements themselves are made available so another researcher can reproduce the comparison end to end.

E. Project Plan

The twelve-week schedule appears in Figure 2. Deploying the instrumented microservice and building the fault injector occupy the first four weeks and conclude with a fixed fault taxonomy aligned to the benchmark's categories, which is the precondition for both comparison legs.

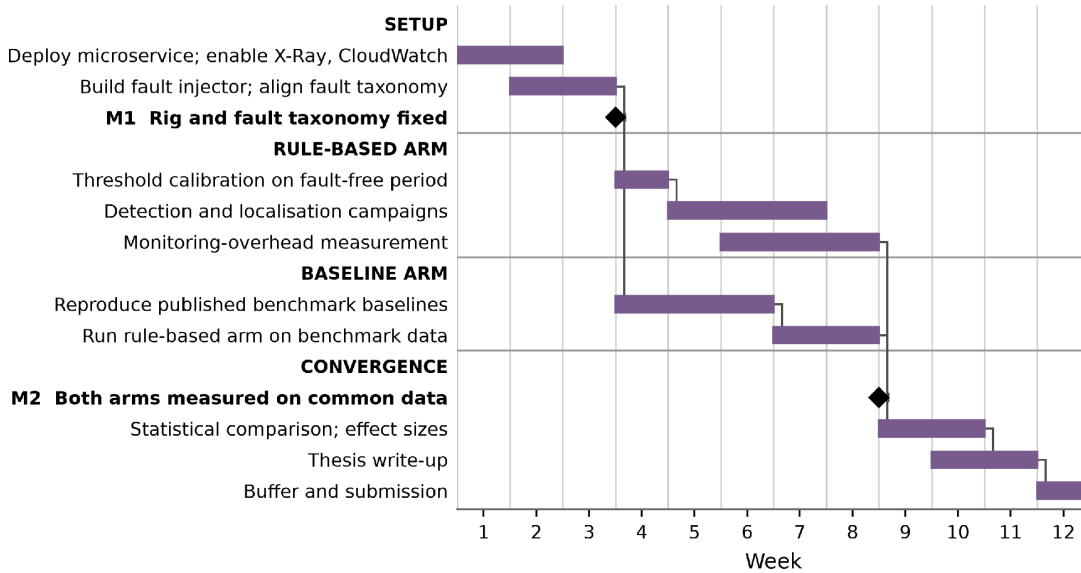


Figure 2 Twelve-week project plan

4. References

- El Khairi, A., Caselli, M., Peter, A. and Continella, A. (2024) 'ReplicaWatcher: training-less anomaly detection in containerized microservices', in *Proceedings of the 2024 Network and Distributed System Security Symposium*. Reston, VA: Internet Society. doi: 10.14722/ndss.2024.24286. [Conference - CORE Ranking A* (ICORE2026)]
- Huang, H., Zhang, X., Chen, P., He, Z., Chen, Z., Yu, G., Chen, H. and Sun, C. (2024) 'TraStrainer: adaptive sampling for distributed traces with system runtime state', *Proceedings of the ACM on Software Engineering*, 1(FSE), pp. 473-493. doi: 10.1145/3643748. [Conference - CORE Ranking A* (ICORE2026)]
- Liu, J., Yang, T., Chen, Z., Su, Y., Feng, C., Yang, Z. and Lyu, M.R. (2023) 'Practical anomaly detection over multivariate monitoring metrics for online services', in *Proceedings of the 2023 IEEE 34th International Symposium on Software Reliability Engineering*. Piscataway: IEEE, pp. 36-45. doi: 10.1109/ISSRE59848.2023.00045. [Conference - CORE Ranking A (ICORE2026)]
- Mertz, J. and Nunes, I. (2023) 'Software runtime monitoring with adaptive sampling rate to collect representative samples of execution traces', *Journal of Systems and Software*, 202, 111708. doi: 10.1016/j.jss.2023.111708. [Journal - JCR Impact Factor 3.8 (2024), Q2]
- Pham, L., Zhang, H., Ha, H., Salim, F. and Zhang, X. (2025) 'RCAEval: a benchmark for root cause analysis of microservice systems with telemetry data', in *Companion Proceedings of the ACM Web Conference 2025*. New York: ACM, pp. 777-780. doi: 10.1145/3701716.3715290. [Conference - The Web Conference, CORE Ranking A* (ICORE2026); Companion track]
- Schmidl, S., Wenig, P. and Papenbrock, T. (2022) 'Anomaly detection in time series: a comprehensive evaluation', *Proceedings of the VLDB Endowment*, 15(9), pp. 1779-1797. doi: 10.14778/3538598.3538602. [Conference - CORE Ranking A* (ICORE2026)]
- Xing, S., Wang, Y. and Liu, W. (2025) 'Multi-dimensional anomaly detection and fault localization in microservice architectures: a dual-channel deep learning approach with causal inference for intelligent sensing', *Sensors*, 25(11), 3396. doi: 10.3390/s25113396. [Journal - JCR Impact Factor 3.5 (2024), Q2]
- Zhang, C., Dong, Z., Peng, X., Zhang, B. and Chen, M. (2024) 'Trace-based multi-dimensional root cause localization of performance issues in microservice systems', in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. New York: ACM, pp. 1-12. doi: 10.1145/3597503.3639088. [Conference - CORE Ranking A* (ICORE2026)]
- Zhang, J. and Yang, H. (2026) 'CPU-only spatiotemporal anomaly detection in microservice systems via dynamic graph neural networks and LSTM', *Symmetry*, 18(1), 87. doi: 10.3390/sym18010087. [Journal - JCR Impact Factor 2.2 (2024), Q2]
- Zhang, W., Yang, Z., Peng, F., Zhang, L., Chen, Y. and Chen, R. (2026) 'GALR: graph-based root cause localization and LLM-assisted recovery for microservice systems', *Electronics*, 15(1), 243. doi: 10.3390/electronics15010243. [Journal - JCR Impact Factor 3.82 (2024), Q2]